ผลงานผู้เรียน · Architecture Decision Record

Lab 7.1 — Feature Scope และ Architecture ของ One-time Voucher

ผลงานผู้เรียน • 18 ตุลาคม 2026 • ADR-VOUCHER-0001 • Status: **Proposed**

เลือก **modular monolith** โดยให้ `VoucherModule` เป็น writer ของ voucher เพียงรายเดียว และ commit state, redemption record, retry result และ outbox ใน transaction เดียว ทั้งหมดเป็น **design proposal — ยังไม่ implement** ส่วน W8 กำหนดขอบเขตเป็น **single-process in-memory fixture**; เอกสารนี้ไม่มี runtime proof

1. แหล่งข้อมูลและสิ่งที่เรียนรู้จาก ADR ต้นฉบับ

อ่าน `task.json`, `prompt.md` และ `team-notes-adr-reference.md` ที่ได้รับแล้ว เอกสาร reference คือ **ADR-0001: บังคับ Team Isolation ที่ ASP.NET Core API** วันที่ 2026-10-13 มี Status: Proposed และระบุ source snapshot ของ Team Notes เป็น branch `main`, commit `158536e` ข้อมูลเกี่ยวกับ source และ tests ในตารางนี้เป็นสิ่งที่ต้นฉบับบันทึกไว้ ไม่ใช่การเปิดตรวจ repository หรือรันทดสอบซ้ำใน Lab นี้

สิ่งที่อ่านพบจริงในโครงต้นฉบับ	สิ่งที่นำมาใช้กับ ADR ของ Voucher
Metadata ระบุ status, date, project, source snapshot และ decision owner/reviewers	ระบุสถานะ Proposed, วันและ owner ของข้อเสนอ Voucher; แยกการอนุมัติออกจากการ implement
Context อธิบาย baseline และปัญหา; Decision drivers ระบุสิ่งที่ต้องรักษา	เริ่มจาก requirement ของ Voucher และ invariants ก่อนเลือก architecture ไม่เริ่มจากชื่อเทคโนโลยี
Options considered เปรียบเทียบข้อดี ต้นทุน และผลพิจารณา; Decision ระบุขอบเขต control	เปรียบเทียบ modular monolith กับ separate service ด้วย concurrency/failure scenarios แล้วกำหนด writer และ transaction boundary
Consequences แยกประโยชน์ ต้นทุน และสิ่งที่ยังไม่แก้; Threat model links ผูกความเสี่ยงกับ control และหลักฐาน	บันทึก coupling, event delivery และ external side effects ที่ยังไม่รับประกัน พร้อมผูกความเสี่ยงของ Voucher กับ verification
Verification and acceptance ระบุว่าไม่ได้รัน security tests; มีทั้ง negative cases และ positive controls	ระบุ expected tests โดยไม่อ้างว่าผ่านแล้ว และตรวจว่าการ redeem ที่ถูกต้องสำเร็จได้ ไม่ใช่ปฏิเสธทุกคำสั่ง
Revisit triggers and lifecycle แยก Proposed → Accepted ออกจาก implementation และใช้ Superseded เมื่อตัดสินใจใหม่	กำหนด trigger และหลักฐานก่อนเปลี่ยน architecture; เก็บเหตุผลเดิมเมื่อมี ADR ใหม่

แยก Team Isolation decision ออกจาก Voucher decision

ประเด็น	Team Notes ADR ต้นฉบับ	ADR ของ Voucher ในผลงานนี้
คำถามที่ตัดสิน	ใครบังคับสิทธิ์ข้อมูลทีม เมื่อใช้ API/DB ร่วมกัน	จัดโครงสร้างและ consistency boundary อย่างไรให้ redeem ได้ไม่เกินหนึ่งครั้ง

ประเด็น	Team Notes ADR ต้นฉบับ	ADR ของ Voucher ในผลงานนี้
ทางเลือกและข้อเสนอ	UI-only, API policy/query scope, PostgreSQL RLS; เสนอ API enforcement ใน deployment เดิม	Modular monolith กับ separate Voucher service; เสนอ monolith ที่มี atomic state/redemption/outbox
สิ่งที่ต้องพิสูจน์	Actor/action/resource policy, team-scoped access และข้อมูลไม่ถูกแก้ข้ามสิทธิ์	Activation prerequisite, terminal/expiry rules, concurrent once-only และ state/event consistency

บทเรียนที่ยืมได้คือ **ไม่เชื่อ UI หรือค่าที่ client อ้าง และไม่ถือว่ามี `Session` แล้ว identity เชื่อถือได้** ต้นฉบับระบุว่า JWT signature/lifetime validation ของ snapshot ยังต้องแก้และทดสอบ ส่วน once-only ต้องใช้ atomic transition กับ uniqueness ของ Voucher เอง: API authorization, BOLA tests, tenant predicates หรือ RLS ไม่ใช่หลักฐานว่า redeem ไม่ซ้ำ เอกสารนี้ไม่คัดข้อสรุป Team Isolation มาทดแทน Voucher decision และไม่อ้างว่าแก้ช่องโหว่ Team Notes แล้ว

---

2. Feature scope และ invariant — design proposal

มีสาม actions: `activate`, `redeem`, `cancel` ใช้ voucher ที่ถูกสร้างไว้แล้ว การออก voucher, payment, inventory และการโอน owner อยู่นอก scope

1. ต้อง activate ก่อน redeem
2. `CANCELLED` หรือหมดอายุแล้ว redeem ไม่ได้
3. ต่อ `voucherId` มี committed redemption ไม่เกินหนึ่งรายการ แม้ request แข่งกันหรือ retry

Success หมายถึงการ commit `ACTIVE → REDEEMED` พร้อม `redemptionId` และ logical `VoucherRedeemed` ครั้งเดียว การคืนผลเดิมเป็น `REPLAY` ไม่ใช่ success ใหม่; จำนวน HTTP responses หรือ event deliveries ไม่ใช่จำนวน redemption

```
ISSUED --activate--> ACTIVE --redeem--> REDEEMED
ISSUED --cancel----> CANCELLED
ACTIVE --cancel----> CANCELLED

count(committed redemptions for voucherId) <= 1
redeem allowed iff authorized AND state = ACTIVE AND now < expiresAt
REDEEMED / CANCELLED เป็น terminal state; ไม่มี reset หรือ re-activate
```

`EXPIRED` เป็น derived condition เมื่อ `now >= expiresAt` จึงไม่ต้องรอ job เปลี่ยน state ใช้ UTC และเวลาฝั่ง server ณจุดตรวจ transition หลังได้ lock. ประวัติ `REDEEMED` ไม่เปลี่ยนเป็น expired ภายหลัง และ target transport retry ที่ยืนยัน commit เดิมอาจคืน `REPLAY` หลัง expiry โดยไม่ consume ใหม่: ทั้งหมดเป็น design proposal. W8 ใช้ integer now และ replay ต้อง reject ตาม §7.

---

3. สาม actions, data owner และ trust — design proposal

ข้อมูลหลักคือ `voucherId`, `issuerId`, `beneficiaryId`, `state`, `expiresAt`, `version` โดย `issuerId` / `beneficiaryId` เปลี่ยนผ่านสาม actions นี้ไม่ได้ Voucher domain team เป็น data owner และ `VoucherModule` เป็น writer ผู้เดียว; Identity team เป็น owner ของ identity/issuer membership

Action / caller	Data และเงื่อนไข	ผล expected / trust assumptions
activate / beneficiary	เข้า voucherId, requestId; อ่าน stored beneficiary, state, expiry; ต้องเป็น ISSUED และยังไม่หมดอายุ	เปลี่ยน ACTIVE, เพิ่ม version และ VoucherActivated; เทียบ trusted subject กับ stored beneficiaryId ไม่รับ owner/state จาก client; ถ้า active แล้วคืน ALREADY_ACTIVE โดยไม่สร้าง event ซ้ำ
redeem / beneficiary	เข้า voucherId, requestId, optional clientReference; อ่าน owner/state/expiry/retry result; ต้อง ACTIVE และยังไม่หมดอายุ	เปลี่ยน REDEEMED, สร้าง redemption/result/VoucherRedeemed อย่าง atomic; key เดิมและ payload เดิมคืน REPLAY, key ใหม่หลัง redeem คืน ALREADY_REDEEMED
cancel / issuer operator	เข้า voucherId, requestId; อ่าน voucher issuer และ trusted issuer permission; ต้อง ISSUED หรือ ACTIVE และยังไม่หมดอายุ	เปลี่ยน CANCELLED, เพิ่ม version และ VoucherCancelled; ไม่มีสิทธิ์ข้าม issuer โดยปริยาย; cancel ซ้ำคืน ALREADY_CANCELLED; cancel หลัง redeem คืน ALREADY_REDEEMED

ตรวจ authorization ก่อนเปิดเผย state; redeem ก่อน activate คืน `NOT_ACTIVE`, หลัง cancel คืน `CANCELLED`, หลัง expiry คืน `EXPIRED` และไม่เขียนข้อมูล การ activate จาก terminal state ถูกปฏิเสธ เมื่อ cancel แข่งกับ redeem ให้ atomic commit เป็นตัวตัดสิน: ผลสุดท้ายเป็น `CANCELLED` พร้อม redemption 0 หรือ `REDEEMED` พร้อม redemption 1 เท่านั้น

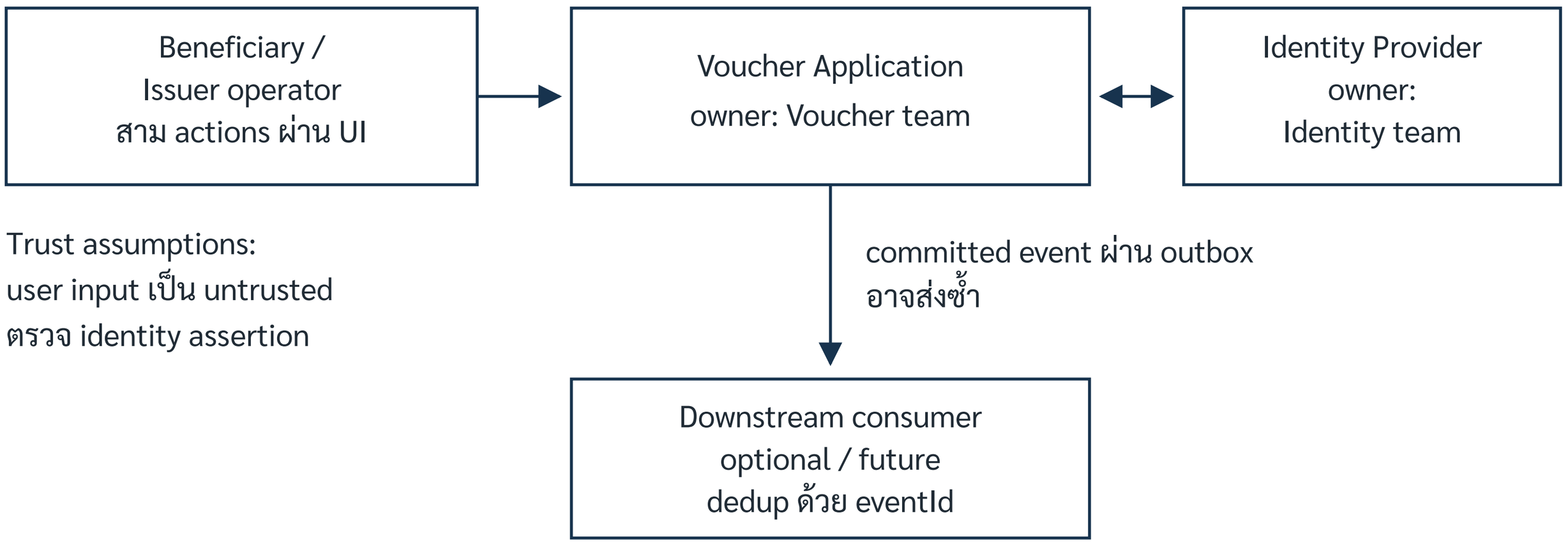
HTTP/auth/session contract — design proposal, ยังไม่ implement

เสนอ `POST /vouchers/{id}/activate`, `/redeem`, `/cancel` เป็น contract สมมติ ไม่ใช่ endpoint ที่ค้นพบหรือทดสอบใน Team Notes UI ส่ง HTTPS พร้อม session cookie; backend ตรวจ session/expiry, CSRF และ permission แล้วสร้าง `ActorContext` จาก trusted identity ไม่ใช้ role/issuer/owner จาก request body เป็นสิทธิ์จริง รายละเอียด session store, credentials และ deployment ยังต้อง implement

เสนอ response: `200` สำหรับ transition/transport replay; `401` session ไม่ถูกต้อง; `404` ไม่พบ/ไม่มีสิทธิ์; `409` state/key conflict; `410` expiry หลัง authorization. เป็น target HTTP proposal ไม่มี HTTP proof. W8 รับ actor เป็น caller label ไม่ทดสอบ auth/role/resource policy และไม่มี transport idempotency.

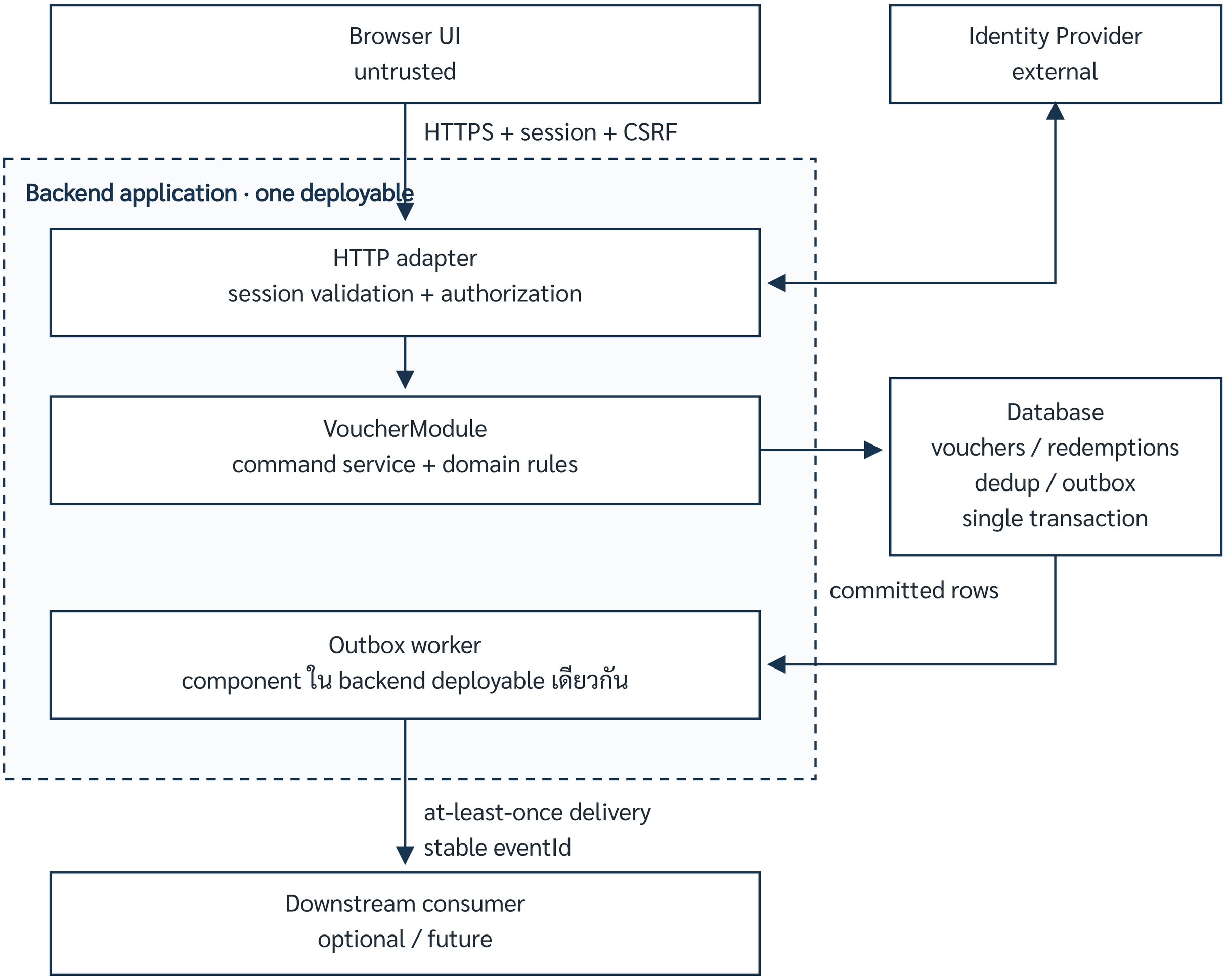
4. Context/container และ ownership boundary — design proposal, ยังไม่ implement

Context



ภาพที่ 1 · Context — ทุกระบบและการเชื่อมต่อเป็น design proposal; ยังไม่ implement

Container



ภาพที่ 2 · Container / components — design proposal ทั้งหมด; กรอบเส้นประคือ backend deployable เดียว

`VoucherModule` และ worker เป็น components ใน backend deployable เดียว ไม่ใช่บริการแยก; database เป็น container แยก Downstream ยังเป็น optional future integration ไม่ได้ implement

Boundary / owner	Trust และข้อบังคับ
Browser → backend / UI team และ Voucher team	Input เป็น untrusted; ซ่อนปุ่มไม่ใช่ authorization; HTTPS ไม่ให้ object permission
Backend ↔ Identity / Identity team	ตรวจ credentials ก่อนสร้าง actor; หากยืนยันสิทธิ์ไม่ได้ให้ปฏิเสธ mutation; จาก adapter เข้า module ยังต้องตรวจ action/resource policy
Module → DB / Voucher team; platform ดูแล infrastructure	Module เป็นเจ้าของ tables และเป็น writer ผู้เดียว; จำกัด credentials; ใช้ transaction, row lock และ unique constraint ไม่สมมติว่า storage สำเร็จเสมอ
Worker → consumer / Voucher team และ consumer team	ส่งซ้ำได้; consumer dedup ด้วย eventId; event ไม่ให้สิทธิ์แก้ voucher และไม่แนบ session token

5. สอง quality scenarios — design proposal / expected, ยังไม่รันทดสอบ

QS-01: Once-only ภายใต้ concurrency

องค์ประกอบ	ข้อกำหนด
Source / stimulus	Beneficiary ส่ง redeem(V-001, R-A) และ redeem(V-001, R-B) พร้อมกันด้วยคนละ requestId; อาจ retry หลัง response หาย
Environment / artifact	Voucher active และยังไม่หมดอายุ; backend อาจหลาย instances ใช้ DB เดียว; artifact คือ command service, voucher row, redemption/outbox
Response	ผู้ชนะ commit ครั้งเดียว; ผู้แพ้ได้ ALREADY_REDEEMED; retry key ของผู้ชนะได้ REPLAY พร้อม redemptionId เดิม
Measure	ทุก interleaving มี committed redemption ไม่เกิน 1; หากมีผู้ชนะต้องมี logical VoucherRedeemed 1 และ final state REDEEMED; หากรอ lock จน expiry ก่อนผ่าน transition check ให้ปฏิเสธ

W8 มี race probe เดิม: alice/bob ส่ง redeem ผ่าน checkpoint/barrier บน service เดียว; ยังไม่รัน และ baseline check/commit ไม่ atomic. Expected คือหนึ่ง success และหนึ่ง reject ไม่ใช่ key-based replay. Target สอง DB connections/instances ยังเป็น future verification นอก W8.

QS-02: Failure ระหว่าง state/event write

องค์ประกอบ	ข้อกำหนด
Source / stimulus	Exception หลัง update state ก่อน insert event; หรือ process หยุดหลัง commit ก่อนตอบ/ส่ง event
Environment / artifact	เริ่ม V-001 = ACTIVE; target ใช้ transactional DB; artifact คือ state, redemption, retry result, outbox และ worker
Response	ก่อน commit ให้ rollback ทุก write; หลัง commit ให้ข้อมูลครบและ retry คืนผลเดิม; worker ส่ง pending event ต่อได้เมื่อกลับมาพร้อม
Measure	ก่อน commit ไม่มี write ใหม่ทั้งชุด; หลัง commit มี redemption/outbox หนึ่งชุดที่สอดคล้องกัน ไม่เกิด REDEEMED ที่ขาด redemption/event และ retry ไม่สร้างชุดที่สอง

W8 ไม่มี draft-copy fault injection หรือ crash recovery probe; QS-02 เป็น future target proposal เท่านั้น ไม่บังคับเพิ่ม fixture/probes. Target ต้องทดสอบ rollback, kill/restart และ duplicate delivery; recovery target ที่ยังไม่วัดคือ drain ≤100 pending events ภายใน 60 วินาทีหลัง dependency พร้อมและ consumer รับ ≥10 events/s ไม่ใช่ SLA ระหว่าง outage.

---

6. ADR-VOUCHER-0001 — เลือก modular monolith

Metadata, context และ decision drivers

Status: Proposed • วันที่: 2026-10-18 • Project: One-time Voucher design / Lab 7.1 • Decision owner/reviewers ที่เสนอ: Voucher domain team ร่วมกับ platform และ identity owners • ยังไม่อนุมัติหรือ implement

ต้องรักษา activation prerequisite, once-only และ state/event consistency ภายใต้ concurrency/failure ตาม QS-01/02 โดยแยก domain ออกจาก transport; W8 ครอบเฉพาะ state/time/repeated consume/race ใน memory ไม่ครอบ QS-02 fault/recovery Constraints คือสมมติว่าทีมเดียวเป็น

owner, ยังไม่มี workload evidence ว่าต้อง scale/release แยก, ไม่มี production deployment ใน Lab และ target storage ต้องรองรับ ACID transaction/row lock/uniqueness สมมติฐานเหล่านี้ต้องยืนยันก่อน implement

Drivers คือ correctness ต่อ voucher เดียว, recovery ที่อธิบายได้, ownership ที่บังคับได้ และ operational cost ที่เหมาะกับ feature ไม่ตัดสินจากจำนวน containers เพียงอย่างเดียว

Options considered — ทั้งสองเป็น design proposal

เกณฑ์	A: Modular monolith	B: Separate Voucher service
Structure / ownership	Module และ worker ใน backend deployable เดียว; module เป็น owner ของ tables	Backend เรียก service ผ่าน network; service เป็น owner ของ DB ของตน ห้าม backend เขียนข้าม
QS-01	Transaction/row lock/unique constraint คุม once-only ข้าม instances ได้	ต้องใช้กลไก atomic/unique เช่นเดียวกัน; การแยก service ไม่ป้องกัน redeem ซ้ำโดยตัวมันเอง
QS-02	State/redemption/result/outbox อยู่ transaction เดียว ไม่มี network hop ระหว่าง adapter กับ domain	Atomic ภายใน service ได้ แต่เพิ่ม timeout/unknown outcome ระหว่าง backend กับ service ต้องส่งต่อ requestId/replay contract
Trust / operations	ต้องรักษา module boundary ด้วย code review และ DB access policy; scale/release/failure ร่วมกับ backend	แยก credentials, scale/release ได้; เพิ่ม service identity, delegated actor, API versioning, monitoring และ retry complexity
ผลพิจารณา	**เลือกเสนอ**: failure boundaries และต้นทุนต่ำกว่าเมื่อ owner เดียว	ยังไม่เลือก: ยังไม่มีเหตุผลด้านทีม/workload ที่ชดเชยต้นทุนเพิ่ม

BFF เป็น adapter pattern และ event-driven เป็น integration pattern ซึ่งอยู่ร่วมกับ A หรือ B ได้ ไม่ใช่คู่ตรงข้ามของ monolith และไม่ได้เลือก broker/deployment เพิ่มในงานนี้

Decision และกลไกที่ต้อง implement

เลือก **A — modular monolith**: ทุก caller ผ่าน public command interface ของ `VoucherModule`; HTTP/identity, clock และ persistence เป็น target adapters แยกจาก domain โดย production เสนอ durable DB. W8 เป็น in-memory VoucherStore กับ public API จริงใน §7 ไม่ใช่ implementation ของ target adapters ทั้งชุด.

Record — owner คือ VoucherModule	ข้อบังคับที่เสนอ
vouchers	voucherId primary key; เก็บ owner/issuer/state/expiry/version; ทุก mutation lock row เดียวกัน
redemptions	redemptionId primary key และ UNIQUE(voucherId); ห้าม reuse voucherId แม้ลบข้อมูลส่วนบุคคล ต้องเก็บ uniqueness marker/tombstone
command_results	Unique (actorId, voucherId, action, requestId); เก็บ fingerprint และผลเดิม; clientReference เป็น correlation text ไม่ใช่สิทธิ์ และรวมใน fingerprint
outbox	eventId คงที่; unique logical event ต่อ voucher/version/type; commit พร้อม state/redemption/result

ตรวจ credentials ที่ adapter → BEGIN → lock voucher row
→ ตรวจ actor/action/resource จาก stored owner/issuer
→ key เดิม + payload เดิม: REPLAY; payload ต่าง: IDEMPOTENCY_CONFLICT

→ ตรวจ state และ server time หลังได้ lock
→ update state/version + insert redemption + retry result + outbox
→ COMMIT → ตอบ transition success
exception ก่อน commit → ROLLBACK ทั้งชุด

จุด linearization คือ commit; activate/cancel ใช้ locking protocol เดียวกันและเขียนเฉพาะ records/events ของ action นั้น หาก connection หายและไม่ทราบ commit outcome ต้อง lookup/retry ด้วย key เดิม ไม่สรุปว่า rollback เอง Worker อ่านเฉพาะ committed outbox และ mark delivered หลัง acknowledgement หากล้มระหว่างส่งกับ mark อาจส่งซ้ำด้วย eventId เดิม

Consequences / trade-offs / สิ่งที่ยังไม่แก้

- ได้ consistency boundary ชัดและทดสอบ domain แยกได้ แลกกับ release, resource และ process failure ที่ร่วมกับ backend; ต้องป้องกัน module อื่นเขียนตารางข้าม boundary
- Lock serialize ต่อ voucher จึงมี contention; ต้องวัด workload ก่อนปรับ ไม่ตัด lock ออกโดยไม่มี consistency strategy ทดแทน
- Outbox เพิ่ม storage/worker/monitoring และ delivery เป็น eventual, at-least-once ไม่ใช่ end-to-end exactly-once
- External payment/fulfillment ต้องมี dedup/idempotency/reconciliation contract เพิ่ม; authorization, deployment, DB recovery และ production readiness ยังไม่ถูกพิสูจน์ด้วย ADR หรือ W8

Risk/control links และ verification

IDs ต่อไปนี้ตั้งสำหรับ Voucher ไม่ใช่ Team Notes threat IDs, SonarQube issue keys หรือผลโจมตี

Risk	Control ที่เสนอ	หลักฐานที่ต้องเก็บ
V-ONCE: concurrent/retried redeem ซ้ำ	Atomic transition, unique redemption, idempotent replay	QS-01; นับ redemption/logical event พร้อม positive success และ rejected duplicate
V-ATOMIC: state กับ event แยกจากกัน	Transactional outbox และ rollback	QS-02; fault ก่อน commit และ recovery หลัง commit
V-ACTOR: caller อ้าง owner/issuer ปลอม	Verified actor + stored resource policy; ไม่รับ owner mutation	Negative/positive policy cases; target ต้องมี HTTP/session tests จริง

Acceptance ยังเป็น **not run**: W8 ตรวจ behavior ใน §7; target ต้องเพิ่ม tests หลาย DB connections/instances, actual rollback/restart, worker duplicate delivery/consumer dedup และ HTTP/auth/session/CSRF ก่อนกล่าวอ้างว่า implement แล้ว ไม่ถือว่า reject ทุกคำสั่งคือผ่าน และไม่นำ Team Notes security test results มาเป็น Voucher proof

Revisit triggers และ lifecycle

Trigger	หลักฐานก่อนเปลี่ยน decision
ต้องมี owner/release/availability แยก	Ownership map, release dependencies และ incident records; ประเมิน separate service
Scale รวมไม่เหมาะหรือ lock contention สูง	Workload, latency/lock-wait และ capacity/cost เทียบ SLO ที่ตกลงจริง; ตรวจ query/index ก่อนแยก service
ต้อง active-active multi-region หรือ storage ไม่มี ACID แบบที่สมมติ	Consistency/failure evidence และ writer routing strategy ใหม่; ห้ามลด once-only โดยไม่มี ADR ใหม่
เพิ่ม external side effects หรือพบ invariant/trust assumption ล้มเหลว	External idempotency contract หรือ counterexample จาก tests; ปรับ protocol ก่อน rollout ต่อ

เมื่อ owner/reviewers อนุมัติจึงเปลี่ยน Proposed เป็น Accepted พร้อมวัน/ผู้อนุมัติ; implementation และ verification ติดตามแยก หากเลือกทางใหม่ให้สร้าง ADR ใหม่และ mark ฉบับนี้ Superseded พร้อมลิงก์ โดยเก็บเหตุผลเดิมตามบทเรียนจากต้นฉบับ

Reference ที่อ่านจริง: Team Notes ADR reference สำหรับโครงสร้างและวินัยด้านหลักฐาน; `task.json` สำหรับ Voucher requirements ไม่ได้เปิด external links ในเอกสาร reference

---

7. W8 handoff — source-confirmed scope, not run

อ่าน baseline `fixture-input/voucher.py`, `voucher_model.py`, `scenario_support.py`, `probes.py` แบบ read-only; ไม่อ่าน instructor reference และไม่รัน. Public API จริง: `issue(code, expires_at)`, `activate(code)`, `cancel(code)`, `redeem(code, actor, now, checkpoint=None)`, `snapshot(code)`, `events()`. `activate/cancel` คืน bool; `redeem` คืน `Redemption(ok, reason, receipt_id, actor)`. Voucher มี `code`, `expires_at`, `state`, `redeemed_by`; states คือ `issued/active/cancelled/expired/used`. `snapshot` เพิ่ม `redemption_count`; event เป็น `{code, actor, receipt_id}` ไม่มี event type, version, CommandResult หรือ outbox.

`actor` เช่น alice/bob เป็น caller label สำหรับบันทึกผู้ redeem ไม่ใช่ authentication, owner, role หรือ issuer permission. `now` เป็น trusted integer argument; NOW=10, EXPIRES_AT=100 ไม่ใช่ injected clock object. Store มี Lock ร่วมทั้ง service; baseline ปล่อย lock ระหว่าง check กับ commit. Race ใช้สอง threads กับ checkpoint/barrier ภายใน child process ที่แยกเพื่อ timeout; ไม่ใช่ distributed/multi-process shared store. ห้ามลบ checkpoint เพื่อหลบ race observation.

Source แสดงช่องว่าง: ตรวจ used เฉพาะ actor ต่างคน, ไม่ guard inactive/expiry และแยก check/commit; นี่เป็น static evidence ไม่ใช่ผล exploit. Baseline มี reasons `redeemed`, `unknown_voucher`, `already_used`; ไม่แต่ง reason ของ inactive/expiry ว่ามีแล้ว. Oracle ที่ต้องการ: non-active/หมดอายุ/repeated consume ต้อง `ok=False`, ไม่มี receipt ใหม่/ไม่มี mutation หรือ event เพิ่ม; positive ต้องทำได้จริง. สำหรับ used ใช้ `already_used` โดยไม่ยกเว้น actor เดิม; **same-voucher replay ต้อง reject ไม่ใช่คืน REPLAY success**.

Probe เดิม / trace	Expected contract — not run
positive / T1 / P1,P2	V-POSITIVE: issue→activate=True→redeem alice ที่ 10; ok=True, reason=redeemed, receipt_id ไม่ว่าง, actor=alice; state=used, redeemed_by=alice, redemption_count=event_count=1
unknown / boundary	V-MISSING: ok=False, reason=unknown_voucher; ไม่มี receipt/event
replay / T2 / P3,P4	V-REPLAY: first สำเร็จ; second โดย alice ที่ 11 ต้อง ok=False, reason=already_used; snapshot คงเดิม, event_count=1
state / T1 / P1	V-STATE แยก service สำหรับ issued และ expired; ทั้งคู่ reject, state คงเดิม, redemption_count=event_count=0
cancel / T1 / P1	V-CANCELLED: activate แล้ว cancel=True; redeem reject, state=cancelled, ไม่มี event
expiry / T1 / P2	V-STATE เริ่ม active, now=100=expires_at; reject, คง active, ไม่มี event; positive ใช้ now=10<100
race / T2 / P4	V-RACE: alice/bob ผ่าน checkpoint; ต้องได้ observations ครบ, checkpoint_calls=2, errors=[], success_count=1, event_count=1; loser ok=False/already_used; state=used, redeemed_by/receipt ตรง winner

Race timeout, exception หรือ observation ไม่ครบเป็น inconclusive ไม่ใช่ผ่าน; exit 0 ของ probes ก็ไม่ใช่ security verdict. เก็บ command ที่ใช้จริง, source revision/hash, clock, outcomes, snapshot, event_count/events และ race completeness เมื่อรันใน W8; ตอนนี้ไม่มีผลรัน. แต่ละ probe สร้าง service ใหม่ ไม่ใช่ reset voucher ที่ใช้แล้ว.

Target V-001/UTC/roles/requestId และ idempotent transport response ยังเป็น **future target design proposal** แยกจาก fixture code/integer clock/reject-repeat. C2 เฉพาะส่วน check-and-transition สัมพันธ์กับ race; C1 auth และ C3 transport retry ไม่ได้ถูกพิสูจน์ด้วย fixture. ไม่กำหนดให้

เพิ่มโปรแกรมหรือ probes: ใช้ positive/unknown/replay/state/cancel/expiry/race ที่มีอยู่. Fault injection, cancel race, DB transaction, durability, outbox/consumer และ HTTP/auth/tenant tests เป็น future target work นอก W8.

8. ตรวจย้อนกลับตาม steps, proof และ aiReviewGuideline

เกณฑ์	จุดตรวจและสถานะ
Step 1: requirements, UI/API/identity/storage/VoucherModule	§2–4 ระบุ invariant, scope, context/container และ labels design proposal; ไม่อ้าง endpoint/DB ของ Team Notes
Step 2: 3 actions, owner/data/trust, HTTP/auth contract	§3–4 ครบ callers, data ownership, trust boundaries; ไม่มี server/scan/exploit
Step 3: อ่าน ADR และเลือก architecture ของ Voucher	§1 อ้างสิ่งที่อ่านจริงและแยกสอง decisions; §5–6 มีสอง quality scenarios, options, constraints, decision, trade-offs และ revisit/lifecycle
Proof: แยก designed application จาก W8	§4 เป็น target proposal; §7 เป็น handoff ของ single-process in-memory fixture ไม่มี HTTP/auth/DB proof
aiReviewGuideline: same voucher / evidence honesty	§7 แยก target V-001 จาก fixture codes และ service ใหม่ต่อ probe; ทุก test เป็น expected/not run ไม่สวม Team Notes/BOLA หรือ in-memory output เป็น runtime proof

9. รายการส่งมอบและข้อจำกัด

`result.md` และ `result.html` เป็นเนื้อหาผลงานเดียวกัน HTML พื้นขาว ตัวอักษรเข้ม หัวเรื่องกรมท่า accent ส้ม กว้าง 1500px และ font หลัก 26px ใช้ Sarabun จาก `file:///home/tanit_pani/mikelopster-workspace/public/fonts/sarabun/Sarabun-Regular.ttf` เท่านั้น ไม่มี embedded font/base64/binary หรือ remote assets; การแสดงฟอนต์ขึ้นกับการเข้าถึง local file ของ browser

อ่าน ADR reference แล้ว แต่ยังไม่มี implementation/runtime proof ของ Voucher หรือ W8 ไม่มี HTTP/auth/DB proof และไม่มีการแก้ training app สถานะทั้งหมดที่ยังไม่ทำคงเป็น design proposal การตรวจครั้งนี้รวม baseline source แบบ read-only แต่ไม่มี runtime proof; ไม่เปิด browser ไม่ใช้ network และไม่ตรวจ repository/tests ของ Team Notes เพิ่ม